

# Etapa 1. Analizador Lexicográfico del lenguaje BOT

Daniela Gragirena. 19-10543

Ricardo Garcia. 20-10274

Daniel Quijada. 20-10518

Septiembre 2026

La implementación de esta cuarta etapa fue desarrollada en Java y utiliza como punto de partida el Árbol Sintáctico Abstracto (AST) generado por el parser. Una vez que el programa supera las verificaciones estáticas, el intérprete recorre este árbol y ejecuta las instrucciones de BOT, manteniendo el estado de los robots, sus valores, posiciones, comportamientos y la memoria asociada al mundo en el cual se desplazan.

La arquitectura de la etapa puede resumirse mediante el siguiente flujo:

Archivo BOT  $\rightarrow$  Lexer  $\rightarrow$  Parser  $\rightarrow$  AST  $\rightarrow$  Análisis de contexto  $\rightarrow$  Intérprete.

El código correspondiente a esta etapa se encuentra en la carpeta **Etapa 4** del repositorio del proyecto.

## 1 Estructura general de la implementación

El archivo `Main.java` coordina las distintas fases del procesamiento.

En primer lugar, el archivo fuente es procesado por el analizador léxico. Si se encuentra un error léxico, la ejecución se detiene inmediatamente. En caso contrario, se vuelve a abrir el archivo y se ejecuta el parser, obteniendo como resultado el AST del programa.

Posteriormente se ejecuta el análisis de contexto:

```
AnalizadorContexto contexto = new AnalizadorContexto();
contexto.analizar(arbol);
```

Si existen errores estáticos, estos se muestran y el programa termina. Si no existen errores, se crea una instancia del intérprete:

```
Interprete motor = new Interprete();
motor.interpretar((Programa) arbol);
```

De esta forma se mantiene una separación clara entre los errores que pueden descubrirse antes de la ejecución y aquellos que dependen del estado dinámico del programa.

## 2 Representación del estado de ejecución

La ejecución de BOT se concentra principalmente en la clase `Interprete`. Esta clase mantiene tres estructuras principales:

```
Map<String, RobotRuntime> robotsEnEjecucion;  
Map<String, Object> variablesLocales;  
Map<String, Object> matrizMundo;
```

El primer mapa almacena los robots definidos en el programa.

El segundo mantiene las variables locales creadas durante la ejecución de los comportamientos.

El tercero representa la matriz bidimensional en la que se desplazan los robots.

### 2.1 Representación de los robots

Cada robot es representado durante la ejecución mediante la clase `RobotRuntime`. Cada instancia mantiene:

- nombre del robot;
- tipo asociado;
- estado activo o inactivo;
- valor almacenado;
- lista de comportamientos;
- coordenada horizontal  $x$ ;
- coordenada vertical  $y$ .

Todos los robots son creados inicialmente en la posición

$$(x, y) = (0, 0),$$

con estado inactivo y sin valor almacenado.

### 2.2 Representación de la matriz

La superficie definida por BOT es conceptualmente infinita, por lo que no resulta conveniente representarla utilizando una matriz convencional.

Por esta razón se utiliza un `HashMap` en el cual sólo se almacenan las posiciones que efectivamente contienen algún valor.

La coordenada se representa mediante una cadena de la forma " $x, y$ ".

Por ejemplo, un valor almacenado en la posición  $(3, -2)$  utiliza como llave "3,-2"

Esta representación evita reservar memoria para posiciones que nunca son utilizadas.

### 3 Interpretación de instrucciones

La ejecución de instrucciones se realiza mediante el método

`correr(Instruccion inst)`

de la clase `Interprete`.

El método identifica dinámicamente el tipo concreto del nodo del AST y ejecuta la semántica correspondiente.

#### 3.1 Secuenciación

Una secuenciación ejecuta primero la instrucción izquierda y posteriormente la derecha:

$$I_1; I_2$$

se interpreta como

$$\text{correr}(I_1) \text{ seguido de } \text{correr}(I_2).$$

#### 3.2 Condicionales

Para un nodo condicional se evalúa inicialmente su expresión booleana.

Si el resultado es verdadero se ejecuta la primera rama. En caso contrario se ejecuta la rama `else`, siempre que exista.

#### 3.3 Iteración

La instrucción `while` se interpreta evaluando repetidamente su condición.

Mientras el resultado sea verdadero se ejecuta el cuerpo:

$$\text{while } B : I$$

se implementa conceptualmente como

$$\text{mientras evaluar}(B) = \text{true}, \text{ correr}(I).$$

#### 3.4 Activación y desactivación

Cada robot posee un atributo booleano que indica si se encuentra activo.

Al ejecutar una activación se verifica previamente que el robot no se encuentre activo. Si ya lo está, se produce un error dinámico.

De manera análoga, al intentar desactivar un robot que ya se encuentra inactivo se produce un error.

### 3.5 Avance de robots

Al ejecutar una instrucción **advance**, el intérprete examina los comportamientos asociados al robot.

Los comportamientos con expresiones booleanas son evaluados en el orden en que aparecen.

El primer comportamiento cuya condición resulte verdadera es ejecutado.

Durante este proceso se utiliza la variable

`robotActual`

para mantener el robot cuyo comportamiento está siendo interpretado. Esto permite implementar correctamente la variable especial **me**.

Si ninguna condición resulta verdadera, se utiliza el comportamiento **default**, en caso de existir.

Si no existe un comportamiento aplicable ni un comportamiento **default**, el intérprete genera un error dinámico de comportamiento inexistente.

### 3.6 Almacenamiento

La instrucción

`store expresion`

evalúa primero la expresión.

Posteriormente se verifica que el tipo obtenido sea compatible con el tipo definido para el robot.

Si existe incompatibilidad se reporta un error dinámico.

En caso contrario, el resultado se almacena en

`robotActual.valorAlmacenado`

### 3.7 Movimiento

Las instrucciones de movimiento modifican las coordenadas del robot actual.

La implementación utiliza las siguientes transformaciones:

$$\begin{aligned}\text{up } n &: y \leftarrow y + n, \\ \text{down } n &: y \leftarrow y - n, \\ \text{right } n &: x \leftarrow x + n, \\ \text{left } n &: x \leftarrow x - n.\end{aligned}$$

Cuando no se proporciona una expresión, se utiliza por defecto una distancia igual a uno.

### 3.8 Drop y Collect

La instrucción **drop** evalúa su expresión y almacena el resultado en la posición actual del robot dentro de **matrizMundo**.

Antes de realizar la operación se verifica la compatibilidad del valor con el tipo del robot.

La instrucción **collect** consulta la posición actual.

Si la celda no contiene un valor, se genera un error dinámico.

Si existe un valor, se verifica su compatibilidad de tipos y luego se almacena:

- en una variable local, si se utilizó **collect as**;
- en el propio robot, en caso contrario.

Después de realizar la colección, la celda es eliminada de la matriz.

### 3.9 Entrada

Para las instrucciones de lectura se utiliza la entrada estándar de Java mediante la clase **Scanner**.

La cadena introducida por el usuario se convierte de acuerdo con el tipo del robot:

- **Integer** para **int**;
- **Boolean** para **bool**;
- carácter individual para **char**.

Si la entrada no puede convertirse al tipo esperado se genera el error de lectura inadecuada.

## 4 Verificaciones dinámicas

Los errores de ejecución se procesan mediante el método

`errorDinamico(String mensaje)`

que imprime el mensaje correspondiente y termina la ejecución del programa.

La implementación incluye verificaciones para:

1. activación ilegal;
2. desactivación ilegal;
3. comportamiento inexistente;
4. división o módulo por cero;
5. lectura inadecuada;

6. colección inadecuada;
7. almacenamiento inadecuado;
8. soltado inadecuado.

La división por cero es comprobada durante la evaluación de expresiones aritméticas, antes de aplicar los operadores de división o módulo.

La compatibilidad de valores con tipos BOT se centraliza mediante el método

```
validarCompatibilidad(Object valor, Tipo tipoBuscado)
```

que establece la correspondencia:

```
INT ↔ Integer,  
BOOL ↔ Boolean,  
CHAR ↔ Character.
```

## 5 Compilación y ejecución

La generación del analizador léxico y sintáctico continúa utilizando JFlex y Java CUP.

El archivo Makefile automatiza la generación de:

- LexerCup.java;
- Parser.java;
- ParserSym.java;

y posteriormente compila los archivos Java del proyecto.

La ejecución del intérprete se realiza mediante el script `bot`, que ejecuta:

```
java -cp ".:java-cup-11b-runtime.jar" Main "$@"
```

De esta forma el programa puede utilizarse mediante:

```
./bot archivo.bot
```